Task Behavior Graphs: A Domain-Driven Model for Profiling Queue Worker Tasks under Uncertain Resource Demand

Anonymous Author(s)

Abstract

Queue-worker systems execute heterogeneous operational work such as replication, repair, cache rebuilds, imports, external-service calls, validation, and media processing. Profiling such work is difficult because the behavior of a task attempt is not determined by task kind alone: intrinsic task features and execution context can change active resource demand, bounded worker occupancy, tail behavior, and the value of additional instrumentation. Existing metrics, traces, profiles, and resource-management models provide useful observations, but they do not by themselves define an attempt-attributed model that separates task identity, intrinsic work, execution context, cost, uncertainty, risk, and profiling policy.

This paper develops Task Behavior Graphs, a domain-driven model for profiling queue-worker task attempts under uncertain resource demand. The model separates task kind, task instance, and task attempt; represents cost through typed resource-demand and worker-occupancy dimensions; treats raw observations as evidence that must be assigned semantic roles; and connects canonical features to cost estimates, reliability diagnostics, risk-relevant consequences, and profiling-policy triggers. The contribution is a formal modeling framework and worked structure for future implementations and evaluations, not an empirical claim about a particular production system.

Keywords: queue workers; task profiling; workload characterization; resource demand; worker occupancy; behavior graphs; observability; instrumentation; admission control; overload regulation.

1 Introduction

Queue-based execution decouples work production from work execution. A producer submits a task instance to a queue, and one or more workers later reserve, execute, retry, acknowledge, or fail that work. Although this pattern is often presented as a background-job abstraction, production queues frequently execute operationally significant work: replication, repair, cache rebuilds, imports, media processing, external-service calls, validation, notification delivery, and other asynchronous operations. In such systems, a worker is an execution boundary rather than merely a message consumer.

This paper studies profiling at the level of logical task attempts inside queue-worker systems. This level is narrower than a worker process, container, node, or queue, but broader than an individual function call or trace span. It is the level at which a concrete execution of a queued task consumes resources, holds bounded execution capacity, produces outcomes, and may be retried or failed. Coarser observations are useful, but they do not by themselves explain which task kind or attempt produced the observed behavior.

The central difficulty is that task behavior is conditional. A task kind and a payload-size bucket may provide a useful baseline, but they do not fully determine resource demand or worker occupancy. Two attempts of the same replication task kind may process similar payloads and still differ because one runs against healthy peers while another runs under degraded peer health, unstable network paths, disk pressure, or retry backoff. The task kind has not changed; the execution context has changed, and that context may affect only selected cost dimensions.

This paper uses a role-specific view of profiling. Task identity and intrinsic features describe the requested work. Execution context describes input-side state around an attempt. Active resource demand and worker occupancy are attempt-attributed cost-side estimates. Observations provide evidence. Uncertainty, risk, and profiling policy are downstream uses of estimates, residuals, and consequences. These roles are related, but they are not interchangeable.

Existing work motivates parts of this view. Cluster managers and fairness policies reason about multiple resource dimensions and observed usage [6, 11]. Workload studies show heterogeneity, dynamic behavior, and task-usage variation at scale [1, 9, 12]. Workload classification systems infer behavior rather than relying only on static resource assumptions [4, 5]. Large distributed systems are often shaped by tail behavior rather than average case alone [2]. Observability systems provide essential metrics, traces, logs, and profiles, but they introduce overhead and require budgeted sampling or instrumentation policies [7, 8, 10]. These lines of work do not remove the need for a queue-worker-specific model that connects logical tasks, execution context, cost estimates, uncertainty, and profiling policy.

This paper proposes *Task Behavior Graphs* as that model. A Task Behavior Graph is a task-kind- or task-family-level representation that connects canonical features to cost, uncertainty, and risk-relevant outputs through explicit influence assumptions. The graph is not treated as proof of causality. It is a structured modeling artifact: it exposes which features are believed to matter, which cost dimensions they influence, and where the model remains uncertain.

The paper makes five contributions. First, it frames queue-worker task profiling as conditional behavior estimation rather than raw telemetry collection. Second, it separates intrinsic task features, execution context, active resource demand, worker occupancy, uncertainty, and operational risk into distinct model roles. Third, it introduces Task Behavior Graphs as a representation for connecting these roles at task-kind or task-family granularity. Fourth, it describes how raw observations can be mapped into canonical features and observed output values through descriptors, manifests, and domain-specific extensions. Fifth, it outlines how profiling policy can be selected from task behavior, uncertainty, consequence, and instrumentation budget.

The remainder of the paper is organized as follows. Section 2 defines the core terminology and notation. Section 3 provides the technical background and motivating distinctions. Section 4 states the profiling problem. Sections 5–8 introduce the cost model, influence domains, task taxonomy, and graph representation. Sections 9–13 discuss feature mapping, profiling methods, observation points, uncertainty, risk, and policy selection. Section 14 describes manifests and domain packs. The remaining sections discuss case studies, evaluation, limitations, related work, and conclusions.

2 Core Terminology and Notation

This section defines only the core terms and notation required to state the problem and introduce the model. The paper uses a larger terminology set, but the complete reference glossary is deferred to Appendix A. The main text therefore avoids front-loading lifecycle details, implementation terms, and domain-specific vocabulary before the problem is stated.

The terminology is role-specific. A term describes one role in the model: identity, intrinsic work, execution context, cost, symptom, outcome, reliability diagnostic, operational risk, or profiling-policy trigger. A single operational event may connect several roles, but that connection does not make the roles equivalent. A raw observation may therefore map to multiple model variables, but each graph node receives one primary influence-domain assignment. This one-domain rule is introduced formally in Section 6.

Section 3 explains the most common confusions; this section fixes the vocabulary and notation used by the remaining sections.

2.1 Core Terms

Table 1: Core terms used in the main model.

Term	Definition
task	A generic unit of work handled by a queue-worker system.
task kind	The semantic type of work submitted to a queue. A task kind selects reusable model structure, relevant features, cost dimensions, graph fragments, and profiling-policy priors, but it does not determine one universal cost value.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more attempts.
task attempt	One concrete execution of a task instance. Resource usage, worker occupancy, observations, outcomes, and residuals are estimated or observed at attempt granularity.
worker	An executor that reserves, executes, retries, acknowledges, discards, or fails task attempts.
task behavior	The broader profile of a task attempt or kind, including cost estimates, symptoms, outcomes, reliability diagnostics, risk-relevant effects, and policy-relevant triggers.
execution context	The input-side environment and state under which a task attempt executes. It may influence cost, symptoms, outcomes, diagnostics, and risk, but it is not itself a cost output.
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a system component. An observation becomes a model variable only after it is mapped to a semantic role.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or domain-specific signals. Canonical features form the input vector available to estimators.
influence domain	A semantic role assigned to a model variable. Influence domains separate task identity, intrinsic work, execution context, resource demand, worker occupancy, symptoms, outcomes, reliability diagnostics, operational risk, and profiling-policy triggers.
cost dimension	A typed dimension of attempt-attributed cost with role, unit, value domain, attribution boundary, observation semantics, and composition semantics.
estimate object	A dimension-level estimate that may contain a point value, quantiles, distributional summaries, conservative envelopes, missingness information, or other diagnostic material.

Term	Definition
resource demand	The modeled active resource need expected for a task attempt along a resource dimension. It is distinct from observed resource usage and environment-side pressure.
resource usage	The observed attempt-attributed use of a resource during execution.
worker occupancy	Bounded execution capacity held by a task attempt over time. Occupancy exists only during intervals in which the capacity is assigned to the attempt.
worker-slot time	The time during which a task attempt holds a worker execution slot.
symptom	An intermediate observed signal that indicates abnormal or relevant behavior but is not itself a final outcome, a cost dimension, or a root-cause claim.
outcome	The completion or lifecycle result of a task attempt, such as success, failure, timeout, cancellation, retryable error, dead-lettering, or lease expiration.
reliability diagnostic	A diagnostic that describes the reliability of a cost estimate or behavior profile, such as residual magnitude, missingness, low sample support, drift, tail ratio, multimodality, or confidence.
uncertainty	Unreliability or unpredictability of a task behavior estimate. In this paper, uncertainty is represented through reliability diagnostics rather than treated as a cost dimension.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Risk is downstream from cost, diagnostics, outcomes, and consequence information.
profiling-policy trigger	A model variable or decision signal that selects or justifies an observation strategy, such as minimal accounting, detailed accounting, tracing, tail-aware profiling, or diagnostic profiling.
Task Behavior Graph	A typed directed graph associated with a task kind or family. It connects canonical features and derived model variables across influence domains and supports estimation of cost, diagnostics, risk, and profiling-policy triggers.
profiling policy	The selected observation and instrumentation strategy for a task kind, attempt, or execution context.

2.2 Notation

Table 2: Notation used throughout the paper.

Symbol	Meaning
$\mathcal{K}$	The set of task kinds.
$\mathcal{I}$	The set of task instances.
$\mathcal{A}$	The set of task attempts.
$i \in \mathcal{I}$	A task instance.
$a \in \mathcal{A}$	A task attempt.
$k(i) \in \mathcal{K}$	The task kind of task instance i.
$i(a) \in \mathcal{I}$	The task instance executed by attempt a.
k_a	A shorthand for $k(i(a))$.
$\mathcal{D}$	The set of influence domains.
D_{id}	The task-identity influence domain.
D_{intr}	The intrinsic-feature influence domain.
D_{ctx}	The execution-context influence domain.
D_{res}	The resource-demand influence domain.
D_{occ}	The worker-occupancy influence domain.
D_{sym}	The symptom influence domain.
D_{out}	The outcome influence domain.
D_{diag}	The reliability-diagnostic influence domain.
D_{risk}	The operational-risk influence domain.
D_{policy}	The profiling-policy trigger influence domain.
$\delta_k : V_k \to \mathcal{D}$	The domain-assignment function for the graph of task kind k.
$\delta(v)$	The influence domain assigned to node v when the task kind is clear from context.
$\rightsquigarrow_{\mathcal{D}}$	The permitted domain-flow relation between influence domains.
$\mathcal{Z}_k$	The canonical input-feature space for task kind k.
$z_a \in \mathcal{Z}_{k_a}$	The canonical input-feature vector available for attempt a.
$\mathcal{X}_k$	The intrinsic-feature space for task kind k.
$\mathcal{C}_k$	The execution-context feature space for task kind k.
P_X^k	The projection $P_X^k : \mathcal{Z}_k \to \mathcal{X}_k$.
P_C^k	The projection $P_C^k : \mathcal{Z}_k \to \mathcal{C}_k$.
$\mathbf{x}_a$	The intrinsic-feature projection $P_X^{k_a}(z_a)$.
$\mathbf{c}_a$	The execution-context projection $P_C^{k_a}(z_a)$.

Symbol	Meaning
Θ_k	The model-state space for task kind k.
$\theta_k \in \Theta_k$	Task-kind-specific model state.
$\mathcal{R}$	The set of active resource-demand dimensions.
$\mathcal{O}$	The set of worker-occupancy dimensions.
$\mathcal{Q}$	The typed disjoint union $\mathcal{R} \dot{\cup} \mathcal{O}$.
$\mathcal{R}_k$	The resource-demand dimensions relevant to task kind k.
$\mathcal{O}_k$	The worker-occupancy dimensions relevant to task kind k.
$\mathcal{Q}_k$	The cost dimensions relevant to task kind k, $\mathcal{R}_k \dot{\cup} \mathcal{O}_k$.
$q \in \mathcal{Q}$	A typed cost dimension.
Δ_q	The descriptor for cost dimension q.
ρ_q	The role of cost dimension q.
u_q	The unit of cost dimension q.
$\mathcal{V}_q$	The value domain of cost dimension q.
α_q	The attribution boundary for cost dimension q.
ω_q	The observation semantics for cost dimension q.
κ_q	The composition operator for cost dimension q.
$\mathcal{E}_q$	The estimate-object space for cost dimension q.
$\widehat{E}_q(a)$	The estimate object for cost dimension q during attempt a.
point_q	Projection from an estimate object to a point estimate.
$\text{quantile}_{q,p}$	Projection from an estimate object to a p-quantile estimate, when supported.
$\widehat{Y}_q(a)$	The point estimate $\text{point}_q(\widehat{E}_q(a))$.
$\widehat{Y}_{q,p}(a)$	The estimated p-quantile for cost dimension q.
$Y_q(a)$	The observed attempt-attributed value for cost dimension q, when available.
$\epsilon_q(a)$	Residual error $Y_q(a) - \widehat{Y}_q(a)$, when $Y_q(a)$ is available.
$\widehat{\mathbf{D}}(a)$	The resource-demand estimate-object vector for attempt a.
$\widehat{\mathbf{U}}(a)$	The worker-occupancy estimate-object vector for attempt a.
$\widehat{\mathbf{C}}(a)$	The structured attempt-level cost estimate $(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a))$.
F_k	The task-kind-specific cost estimator for task kind k.
$F_{k,q}$	The dimension-specific estimator for task kind k and dimension q.
$\widehat{B}_q$	The estimated intrinsic baseline function for dimension q.

Symbol	Meaning
$\widehat{A}_q$	The estimated contextual amplification factor for dimension q.
$\widehat{O}_q$	The estimated additive overhead function for dimension q.
$\widehat{\mathbf{C}}_{\text{base}}(a)$	The cost estimate excluding additional profiling overhead.
$\widehat{\mathbf{C}}_{\rho}(a)$	The cost estimate including overhead from profiling policy ρ.
$\mathcal{G}_k$	The Task Behavior Graph associated with task kind k.
V_k	The node set of $\mathcal{G}_k$.
E_k	The edge set of $\mathcal{G}_k$.
$v \in V_k$	A graph node.
$e \in E_k$	A graph edge.
$\tau(v)$	The node type assigned to node v.
$\lambda(e)$	The edge role assigned to edge e.
w_e	The estimated sensitivity, weight, or parameter associated with edge e.
γ_e	The confidence value associated with edge e.
$\text{Diag}(a)$	Reliability diagnostics for attempt a.
h_{diag}	A diagnostic function that maps cost estimates, residuals, features, and model state to a diagnostic object.
$S(a)$	Consequence information for attempt a, such as tenant criticality, dependency fragility, or queue-starvation impact.
$\text{Risk}(a)$	The operational-risk estimate for attempt a.
g_{risk}	A risk function that maps cost estimates, diagnostics, consequence information, and model state to operational risk.
$\mathcal{P}$	The set of available profiling policies.
B_{obs}	The observation budget available to profiling-policy selection.
J_p	A policy score or rule predicate for profiling policy p.
$\rho(a)$	The profiling policy selected for attempt a or a comparable class of attempts.

3 Background and Motivation

This section provides the technical background needed for the problem statement. It does not introduce the full model. Its purpose is to explain why queue-worker task profiling cannot be reduced to process-level monitoring, queue-level metrics, or generic tracing. The section

first identifies the task attempt as the relevant execution unit. It then separates the roles that are most often conflated: attempt-side resource demand, attempt-side worker occupancy, environment-side execution context, observed symptoms, and downstream risk. Finally, it motivates feature mapping and budgeted profiling.

3.1 Queue-Worker Execution and Task Attempts

Queue-worker systems execute work through a lifecycle that usually includes enqueue, reserve or dequeue, start, processing, retry or backoff when needed, commit or output write, and acknowledgement or failure. A single task instance can produce multiple attempts because of retry policy, lease expiration, timeout, cancellation, worker failure, or rescheduling. Each attempt may execute under a different worker state, dependency state, retry state, or domain state.

The attempt is therefore the most precise unit for attributing execution behavior. A task kind describes the semantic type of requested work. A task instance describes one queued unit of that kind. A task attempt describes one concrete execution of that instance under one execution context. Resource usage, worker occupancy, outcome, and residual error are usually measured or estimated most cleanly at this boundary. Aggregating only by queue, worker process, or node can reveal pressure, but it can hide which task kind, attempt, or context produced the observed behavior.

A worker may hold capacities that are not visible as high active utilization. During an attempt it may hold a worker slot, task lease, dependency connection, lock, semaphore permit, or tenant-level concurrency slot. These held capacities are occupancy outputs attributed to the attempt. They are different from the surrounding worker or worker-pool state. For example, a worker pool may already be close to saturation before an attempt starts; that pool state is execution context. If the attempt then holds one worker slot for thirty seconds, that duration is worker-slot occupancy attributed to the attempt.

3.2 Resource Demand, Worker Occupancy, and Context Pressure

Distributed resource-management systems commonly reason about multiple resource dimensions rather than a single scalar load. Cluster managers and allocation policies account for dimensions such as CPU, memory, storage, and other capacity constraints when placing, admitting, or allocating work [6, 11]. Queue-worker profiling inherits this multi-resource setting, but adds a task-attribution problem: the model must distinguish active resource demand, worker occupancy, and context pressure.

Active resource demand describes the modeled active resource need of the attempt itself, such as CPU time, memory peak, allocation volume, disk reads and writes, network transfer volume, or accelerator use. Worker occupancy describes bounded execution capacity held by the attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant concurrency time. Context pressure describes the surrounding state that may influence the attempt, such as CPU saturation, memory pressure, disk queue depth, network instability, dependency saturation, connection-pool pressure, or worker-pool utilization.

These roles are related by influence, not by identity. A degraded dependency may increase dependency wait time. That wait may increase worker-slot time if the slot remains held during the wait. The same degradation may also increase retry count or wall-clock duration. In this chain, dependency health is a context factor, dependency wait time is an observed intermediate duration, retry count and wall-clock duration are symptoms or outcomes, and worker-slot time is an occupancy cost only when the slot remains held. Treating all of them as the same quantity would obscure attribution and double-count one underlying degradation.

Demand must also be separated from pressure. CPU demand is not CPU pressure; disk I/O demand is not disk pressure; network transfer volume is not network instability; worker-slot time is not worker-pool pressure. Demand and occupancy are attempt-attributed quantities. Pressure is an input-side state of the environment. A profiling model should preserve these roles before projecting them into any score used by a scheduler, admission controller, or profiling policy.

3.3 Role Assignment for Observations

Observations do not carry model roles by themselves. A raw metric, span, log record, profile sample, or domain event may become an input feature, an observed output value, a symptom, an outcome, or risk evidence depending on what it measures and how it is attributed.

For example, a packet-loss metric may be mapped to a network-instability context feature. Worker start and finish events may be mapped to observed worker-slot time if they delimit the interval during which a worker slot is held. A timeout log may be treated as a symptom or outcome of a dependency call. A sustained increase in queue delay may be risk-relevant evidence. These mappings are semantic choices; they are not implied by metric names alone.

This role assignment prevents double counting. Network degradation, dependency wait, retry count, wall-clock duration, and worker-slot time may all co-occur in the same incident. A model should not treat all of them as independent root causes. It should represent which signal describes context, which signal is an intermediate symptom, which signal is an observed cost dimension, and which downstream effect is a risk-relevant consequence. Section 6 later formalizes these roles as influence domains.

3.4 Workload Variability and Context Dependence

Production workloads are heterogeneous and dynamic. Prior studies of large compute clusters and co-located workloads show substantial variation in task usage, infrastructure conditions, and workload behavior across time, machines, and task populations [1, 9, 12]. Workload classification systems such as Paragon and Quasar similarly motivate inferring behavior from observed properties and performance constraints rather than relying only on static declarations or user-provided reservations [4, 5]. For queue workers, the implication is direct: task kind, queue name, and declared size can provide a baseline, but not a complete behavior model.

A replication task illustrates the issue. Two attempts may have the same task kind and similar payload size, but execute against different replica sets. Under stable network conditions and healthy peers, the attempt may complete near its baseline. Under degraded peer health, packet loss, high replication lag, disk pressure, or retry backoff, the same logical operation may produce larger worker-slot time or connection hold time. The task did not become a different task kind. Its execution context changed, and that context amplified selected cost dimensions.

Other task families expose different sensitivities. A transformation task may be mostly shaped by record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. An external-service task may be shaped by dependency health, fan-out width, timeout policy, connection-pool availability, and retry state. A repair, compaction, cache rebuild, or migration task may be shaped by hidden domain state such as fragmentation, dirty data volume, replica divergence, cache warmth, or disk queue depth. A useful profiling model must therefore ask which features are relevant for a task kind rather than impose one global metric set on all tasks.

Average behavior is not enough. Large-scale distributed systems are often shaped by tail behavior and stragglers [2, 3]. In a queue-worker system, rare attempts that hold worker slots, leases, locks, or dependency connections for unusually long periods can reduce effective capacity and increase queueing delay even when mean duration appears acceptable. This motivates estimates that preserve distributional behavior when tails affect control decisions.

3.5 Profiling Overhead and Observation Policy

Observation has cost. Instrumentation, tracing, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export can add CPU work, allocations, memory pressure, storage volume, and network traffic. Dapper was designed with low overhead as a central concern, and OpenTelemetry treats sampling and telemetry performance as operational constraints [7,8,10]. Collecting more observations is therefore not automatically better.

The right observation strategy depends on the task kind, uncertainty, expected consequence, and instrumentation budget. Stable, low-consequence tasks may need only minimal accounting. Unknown, high-tail, context-sensitive, or high-consequence tasks may justify detailed accounting, sampled tracing, tail-aware capture, or diagnostic profiling. The instrumentation used to understand a system should not become an unbounded source of work or cardinality.

3.6 Motivating Gap

The preceding points expose a gap between available observations and the model needed for queue-worker task profiling. Resource-management systems reason about multiple resources; workload studies show variability and context dependence; observability systems expose metrics, traces, logs, and profiles. These pieces do not by themselves specify how to attribute behavior to logical task attempts, separate intrinsic demand from execution context, account for worker occupancy, assign semantic roles to observations, or choose instrumentation under an overhead budget.

The gap becomes clearer when tasks are compositional. A single task kind may include CPU processing, network transfer, dependency calls, retries, commits, and cleanup. Some quantities compose by summation, others by maximum, critical path, retry expansion, or occupancy-specific rules. Without an explicit model of metric roles and composition semantics, the system cannot reliably combine component behavior into an attempt-level estimate.

The next section states this gap as a profiling problem. Later sections define the cost model, influence domains, task taxonomy, Task Behavior Graphs, feature mapping, and profiling-policy selection as a structured response.

4 Problem Statement

The preceding section motivates a gap between available observations and the model needed to profile logical task attempts in queue-worker systems. This section states the problem addressed by the paper. It defines the system model, available information, expected outputs, main constraints, assumptions, and non-goals. The purpose is to state the problem without introducing the full model that later sections define.

At a high level, the problem is to estimate the behavior of a task attempt from its task identity, intrinsic task features, execution-context information, task-kind model state, and available history while respecting profiling overhead. The output is structured. It includes cost estimates, reliability diagnostics, risk interpretation, and a profiling-policy recommendation.

4.1 System Model

We consider queue-worker systems in which producers submit task instances to one or more queues and workers execute task attempts. A task kind identifies the semantic type of work requested. A task instance is one queued unit of that kind. A task attempt is one concrete execution of a task instance. A single task instance may produce multiple attempts because of retries, lease expiration, timeouts, cancellation, worker failure, or rescheduling.

The profiling boundary is the task attempt. This boundary is narrower than a worker process, container, node, or worker pool, but broader than a single function call or trace span. An

attempt may include sequential, parallel, conditional, retried, or partially overlapping phases. The paper does not assume that every phase is perfectly instrumented.

Each task kind may have task-kind-specific model state. This state may contain declared descriptors, learned parameters, historical summaries, calibration data, quantile sketches, residual statistics, or domain rules. The model state is updated or validated from historical observations, but the paper does not require one specific learning algorithm.

Each attempt executes under an execution context. Execution context is the input-side state surrounding the attempt; it is not the cost of the attempt. It may include worker state, worker-pool pressure, node pressure, dependency health, network condition, disk pressure, cache state, replica state, tenant pressure, retry state, and domain-specific hidden state. Some context factors are directly observable, some are derived from raw observations, and some remain hidden.

The model is task-kind-specific. Different task kinds may require different features, cost dimensions, observation points, composition rules, and profiling policies. The problem therefore cannot assume one global feature set or one global metric list that is equally relevant to all task kinds.

4.2 Available Information

For an attempt a, the profiler may have access to five classes of information.

First, task identity describes what is being executed. It may include the task kind, task version, queue, producer, tenant, priority, deadline, and attempt number. Task identity selects the relevant task-kind-specific model; it is not by itself a cost estimate.

Second, intrinsic task features describe work contained in the task instance. Examples include payload size, record count, object count, partition size, batch size, input format, operation mode, declared resource hints, or domain metadata. These features may be available before execution and can provide a baseline for expected work.

Third, execution-context information describes the environment in which the attempt is executed or expected to execute. Examples include worker-pool pressure, node CPU or memory pressure, disk queue depth, network instability, dependency health, connection-pool pressure, cache warmth, replica health, replication lag, tenant quota usage, and retry state.

Fourth, task-kind model state describes reusable information associated with the task kind or task family. It may include descriptors, estimator parameters, default priors, historical distributions, residual statistics, confidence values, or domain-specified rules.

Fifth, historical observations describe previous attempts of the same or related task kinds. They may include observed resource usage, observed worker occupancy, durations, retry counts, outcomes, residuals, tail quantiles, or drift indicators. Historical data may be unavailable for new task kinds, new versions, rare contexts, or recently changed workloads.

Raw observations are not assumed to be model-ready. Metrics, logs, traces, runtime profiles, and domain events must be interpreted, normalized, and mapped into canonical input features, observed output values, symptoms, outcomes, or risk evidence before the model can use them.

4.3 Desired Outputs

The profiling problem produces four classes of outputs.

The first output class is cost estimates. These include active resource-demand estimates over relevant resource dimensions and worker-occupancy estimates over relevant occupancy dimensions. Cost estimates may be point estimates, distributional estimates, or conservative envelopes.

The second output class is reliability diagnostics. These include residuals, uncertainty indicators, confidence values, drift indicators, missing-feature signals, insufficient-history indicators, and tail-behavior summaries. These diagnostics describe how reliable the cost estimates are; they are not themselves active resource demand or worker occupancy.

The third output class is operational-risk interpretation. Risk is not the same as task cost. It captures the consequence of underestimating, misclassifying, or mishandling an attempt.

The fourth output class is a profiling-policy recommendation. Given the task kind, context, reliability diagnostics, risk, and profiling budget, the system should determine what observation strategy is justified. The policy output is a profiling decision, not a final scheduling, routing, or placement decision.

4.4 Core Challenges

The first challenge is conditional behavior. Attempts with the same task kind and similar intrinsic features may behave differently under different execution contexts. A model based only on task kind, queue, or payload bucket can provide a baseline, but it cannot explain context-dependent amplification, suppression, or drift.

The second challenge is multidimensionality. Active resource demand, bounded worker occupancy, reliability diagnostics, and operational risk have different meanings and units. Combining them prematurely into one scalar hides information needed by downstream control layers.

The third challenge is typed cost semantics. CPU time, memory peak, network bytes, worker-slot time, and lock hold time do not compose in the same way. Some dimensions sum, some take maxima, some are interval measures, and some follow critical-path or retry-expansion semantics.

The fourth challenge is attribution. Process-level, container-level, node-level, and queue-level observations are often easier to collect than task-attempt-level observations. Aggregate observations may indicate pressure, but they do not automatically identify which task kind, instance, attempt, phase, context factor, or dependency produced the observed behavior.

The fifth challenge is semantic interpretation. Raw observations do not state their model role. Without explicit semantic roles, a model may double-count related signals or treat symptoms as independent causes.

The sixth challenge is compositional behavior. A task kind may include CPU processing, network transfer, dependency calls, retries, commits, and cleanup. The profiling problem must preserve enough structure to combine component behavior consistently.

The seventh challenge is partial observability. Important influences may be missing, stale, hidden, or domain-specific. The model cannot assume that all relevant state is observable.

The eighth challenge is profiling overhead. Instrumentation is not free. A profiling model must decide not only what would be useful to observe, but what is worth observing under an overhead budget.

4.5 Scope and Assumptions

This paper assumes that task attempts can be identified at least by task kind and attempt boundary. It assumes that some task metadata or intrinsic features may be available before or during execution, that execution-context information may be available but incomplete, and that profiling decisions are useful even when final scheduling, routing, placement, or admission decisions are made by another layer.

The paper also assumes an open-world metric environment. No core system can know all possible domain-specific metrics in advance. The model must therefore allow raw observations to be mapped into canonical features and must keep missing or unexplained influences explicit instead of hiding them behind a fixed global metric list.

4.6 Non-Goals

This paper does not propose a complete distributed scheduler. It does not define a placement algorithm, routing algorithm, gossip protocol, load-balancing policy, or cluster-wide resource allocator. Such systems may use the estimates produced by task profiling, but they are not the object of this paper.

This paper does not replace metrics, logs, traces, runtime profiles, or existing observability tools. These systems provide observations. The problem addressed here is how to interpret observations for task-kind-specific profiling and how to decide which observations are justified.

This paper does not claim that Task Behavior Graphs prove causality. A graph can make modeled influence assumptions explicit, but causal validation requires additional evidence, experiments, interventions, or domain knowledge. The graph is a profiling model, not a causal proof.

This paper does not attempt to define a globally complete taxonomy of all workload metrics, nor does it require exact prediction for arbitrary tasks. The goal is to produce structured estimates, expose uncertainty, and guide profiling policy.

Finally, this paper does not specify a full production implementation of a package manager, expression language, runtime virtual machine, or cross-language manifest compiler. Later sections discuss manifests and implementation considerations only to the extent needed to make the proposed model concrete.

5 Task Cost Model

Section 4 states the profiling problem as a structured estimation problem. The profiler may observe task identity, intrinsic task features, execution context, task-kind model state, and historical behavior, and it should produce cost estimates, reliability diagnostics, risk interpretation, and profiling-policy recommendations. This section defines only the first of those output classes: attempt-attributed cost estimates.

The cost model is intentionally narrower than the complete task-behavior model. It does not define influence domains, graph topology, feature mapping, uncertainty metrics, risk scoring, or profiling-policy selection. Those layers consume or organize cost information, but they are not part of the cost object itself. The purpose of this section is to define what a cost estimate is, what it is estimated over, how it is typed, how it is related to observed values, and why a single scalar load value is insufficient.

The section proceeds in the order in which a reader needs the concepts. First, it defines the attempt-level cost object. Second, it separates active resource demand from worker occupancy. Third, it defines the feature inputs and task-kind-specific model state used by estimators. Fourth, it introduces typed cost dimensions and dimension-level estimate objects. Fifth, it defines dimension-specific estimators and the common baseline–amplification–overhead decomposition. Finally, it explains overhead, observations, residuals, distributional estimates, and the boundary between the cost model and the later graph, diagnostic, risk, and policy layers.

5.1 Attempt-Level Cost Object

The unit of cost attribution is the task attempt. A task kind defines reusable model structure: which features are relevant, which cost dimensions are estimated, and which estimators or graph fragments may be used. A task instance supplies the logical work to be performed. A task attempt supplies the concrete execution under one execution context. Therefore, structure is task-kind-level, but evaluation is attempt-level.

Let $a \in \mathcal{A}$ be a task attempt. The attempt executes task instance $i(a) \in \mathcal{I}$, and the task

kind of that instance is $k(i(a)) \in \mathcal{K}$. For readability, write

$$k_a = k(i(a)).$$

The symbol k_a does not introduce a new entity. It is only a shorthand for the kind of the task instance executed by attempt a. This shorthand is useful because all task-kind-specific objects used below, such as relevant dimensions, feature projections, and estimator state, are selected by k_a.

The attempt-level cost estimate is a structured object:

$$\widehat{\mathbf{C}}(a) = \left(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a)\right).$$

The first component, $\widehat{\mathbf{D}}(a)$, contains estimates for active resource-demand dimensions. The second component, $\widehat{\mathbf{U}}(a)$, contains estimates for worker-occupancy dimensions. The hat indicates that this is a model estimate rather than an observed value. The bold symbols indicate that the estimate is vector-valued or structured rather than scalar.

For task kind k, let $\mathcal{R}_k$ be the set of resource-demand dimensions that the model estimates for that kind, and let $\mathcal{O}_k$ be the set of worker-occupancy dimensions that the model estimates for that kind. For attempt a, the two components are the dimension-indexed families

$$\widehat{\mathbf{D}}(a) = \left\{\widehat{E}_r(a) \mid r \in \mathcal{R}_{k_a}\right\}, \qquad \widehat{\mathbf{U}}(a) = \left\{\widehat{E}_o(a) \mid o \in \mathcal{O}_{k_a}\right\}.$$

The elements of these families are estimate objects, not only numeric point values. This distinction matters because the model may need to preserve a point estimate, tail quantiles, conservative envelopes, missingness information, and other diagnostic material for the same cost dimension. A downstream policy can choose which projection of an estimate object it needs, but the cost model should not discard the rest of the information prematurely.

This representation also explains why task cost is not modeled as a universal scalar. CPU seconds, memory bytes, network bytes, slot-seconds, lock-seconds, and connection-seconds have different units and different composition rules. A single score can be derived later by a policy, for example

$$s_\rho(a) = \pi_\rho\left(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a)\right),$$

where π_ρ is a policy-specific projection. However, $s_\rho(a)$ is not the cost model. It is a decision-specific view of the structured cost estimate.

The execution context is not a third component of $\widehat{\mathbf{C}}(a)$. It is an input-side condition used by estimators. For example, degraded peer health may amplify the estimated worker-slot time of a replication attempt, but the resulting value is still an estimate for the worker-slot-time dimension. Treating peer health itself as a cost output would mix context with attempt-attributed cost and would make later attribution and risk interpretation ambiguous.

As a running example, consider a replication task kind. The task kind may define resource-demand dimensions such as network output bytes and disk read bytes, and worker-occupancy dimensions such as worker-slot time and connection hold time. Two attempts of this kind may use the same model structure, but they can produce different estimate objects because one attempt runs with healthy peers and a small payload while another runs with degraded peers, packet loss, and retry state. The task kind selects the model; the attempt determines the evaluated cost.

5.2 Resource Demand and Worker Occupancy

The cost object separates two families because they answer different questions. Resource demand asks how much active resource use is expected or observed for the attempt. Worker occupancy

asks how much bounded execution capacity is held by the attempt over time. These quantities may be correlated, but they are not interchangeable.

A resource-demand dimension describes active resource work attributable to the attempt. Examples include CPU time, allocation volume, memory peak, disk read bytes, disk write bytes, network input bytes, network output bytes, or accelerator time. For a resource dimension $r \in \mathcal{R}_{k_a}$, the estimate object is $\widehat{E}_r(a)$, and its point projection is

$$\widehat{Y}_r(a) = \mathrm{point}_r\Big(\widehat{E}_r(a)\Big).$$

The interpretation of $\widehat{Y}_r(a)$ depends on the descriptor of dimension r. For CPU time, it may represent attributed CPU seconds. For network output, it may represent attributed bytes. For memory peak, it may represent a maximum rather than a sum. Therefore, a resource-demand dimension is not defined only by a metric name; it is defined by its descriptor, attribution boundary, observation semantics, and composition operator.

Worker occupancy describes bounded capacity held by an attempt. Examples include worker-slot time, lease time, connection hold time, lock hold time, semaphore-permit time, and tenant-concurrency time. For an occupancy dimension $o \in \mathcal{O}_{k_a}$, the estimate object is $\widehat{E}_o(a)$, and its point projection is

$$\widehat{Y}_o(a) = \mathrm{point}_o\Big(\widehat{E}_o(a)\Big).$$

The unit of an occupancy dimension is usually a capacity-time unit, such as slot-seconds, connection-seconds, lock-seconds, or permit-seconds. These units make explicit that occupancy is not merely elapsed time. It is elapsed time weighted by the amount of bounded capacity held by the attempt.

The interval semantics of occupancy can be written explicitly. Let $\mathcal{T}_{o,a}$ be the set of time intervals during which the capacity associated with occupancy dimension o is held by attempt a. If an attempt can hold more than one unit of that capacity, let $m_o(t, a)$ denote the amount of capacity held at time t. When the required interval observations are available, the observed occupancy value can be understood as

$$Y_o(a) = \int_{\bigcup \mathcal{T}_{o,a}} m_o(t, a)\, dt, \qquad o \in \mathcal{O}_{k_a}.$$

For a single worker slot, $m_o(t, a)$ is usually 1 while the slot is held and 0 otherwise. For semaphore permits, $m_o(t, a)$ may be the number of permits held. For dependency connections, it may be the number of connections held by the attempt. The formula is not a required implementation. It states the model semantics: occupancy is capacity held over time, not wall-clock duration by itself.

This distinction avoids a common error. Suppose an attempt waits on a degraded dependency for thirty seconds. The dependency degradation is execution context. The dependency wait is an observed waiting signal or symptom. The attempt's wall-clock duration is an elapsed-time outcome. Worker-slot time increases only if the worker slot remains held during the wait. Connection hold time increases only if a dependency connection remains held during the wait. Thus, the same incident may affect several variables, but only some of them are cost dimensions.

The separation also explains why active resource demand alone is insufficient. A CPU-heavy transformation may use substantial CPU time and allocation volume while holding a worker slot for a short interval. A dependency-bound task may use little CPU while holding a worker slot or connection for a long interval. Both attempts can be costly, but they consume different kinds of capacity and should be represented by different dimensions.

5.3 Feature Projections and Model State

The cost model is evaluated from attempt-level input features and reusable task-kind state. Let $z_a \in \mathcal{Z}_{k_a}$ be the canonical input-feature vector available for attempt a after feature mapping

and normalization. The vector may be partial and may include explicit missingness indicators. It contains information available to the estimator before or during the attempt. It does not contain post-execution observed output values for the same attempt.

The canonical input vector is separated into role-specific projections:

$$\mathbf{x}_a = P_X^{k_a}(z_a), \qquad \mathbf{c}_a = P_C^{k_a}(z_a).$$

The projection $\mathbf{x}_a \in \mathcal{X}_{k_a}$ contains intrinsic task features. These describe the work contained in the task instance: payload size, record count, object count, batch size, partition size, operation mode, or domain metadata. The projection $\mathbf{c}_a \in \mathcal{C}_{k_a}$ contains execution-context features. These describe the environment around the attempt: worker-pool pressure, node pressure, dependency health, network condition, disk pressure, cache state, retry state, tenant pressure, or domain-specific hidden state when it is available.

The projections are task-kind-specific because different kinds need different features. A compute-transform task may project record count and algorithm mode into $\mathbf{x}_a$, while a replication task may project payload size, replica count, and divergence level. The same replication task may project peer health, network condition, disk pressure, and retry state into $\mathbf{c}_a$. A global feature vector may exist in an implementation, but the mathematical model uses task-kind-specific spaces to avoid assuming that one fixed metric list explains all task behavior.

For each task kind k, the model may maintain task-kind-specific state $\theta_k \in \Theta_k$. This state is reusable across attempts. It may contain dimension descriptors, learned parameters, historical summaries, calibration values, quantile sketches, residual statistics, priors, or domain rules. It may be initialized from a manifest, refined from historical observations, and revised when residuals indicate drift, missing features, or model error.

The roles of the three objects are different:

$$z_a \quad \text{is attempt-level input,}$$

$$\theta_{k_a} \quad \text{is reusable task-kind state,}$$

and

$$Y_q(a) \quad \text{is an observed output value, when available.}$$

This separation prevents leakage between estimation and observation. A value observed after attempt a may update θ_{k_a} for future attempts, but it is not an input feature for estimating the same attempt before it runs.

5.4 Typed Cost Dimensions

A cost dimension is not merely a metric name. It is a typed descriptor that defines the role, unit, attribution, observation, and composition semantics of a quantity. This is necessary because superficially similar measurements may have different meanings. For example, elapsed wall-clock time, worker-slot time, connection hold time, and dependency wait time are all durations, but they do not describe the same model role.

The descriptor is part of the model definition for a dimension. It is not an observed runtime value and is not inferred from a single attempt. In an implementation, a descriptor may be supplied by a core dimension registry, a Task Behavior Manifest, or a domain-specific extension. The descriptor states how values of the dimension are typed, attributed to attempts, observed from raw telemetry, and composed across phases, intervals, or retries.

Let

$$\mathcal{Q} = \mathcal{R} \dot{\cup} \mathcal{O}$$

be the typed disjoint union of resource-demand and worker-occupancy dimensions. For each $q \in \mathcal{Q}$, define a descriptor

$$\Delta_q = (\rho_q, u_q, \mathcal{V}_q, \alpha_q, \omega_q, \kappa_q)\,.$$

Here ρ_q is the role of the dimension, u_q is the unit, $\mathcal{V}_q$ is the value domain, α_q is the attribution boundary, ω_q is the observation semantics, and κ_q is the composition operator. Membership in $\mathcal{R}$ or $\mathcal{O}$ determines the dimension family, but the explicit role field ρ_q remains useful in descriptors, manifests, registries, and validation rules because it makes the intended role machine-checkable.

Each field prevents a specific modeling error. The role ρ_q prevents mixing active resource demand with worker occupancy. The unit u_q prevents combining incompatible quantities, such as bytes and seconds. The value domain $\mathcal{V}_q$ defines admissible values, usually non-negative values with a unit. The attribution boundary α_q states when a value belongs to a particular attempt. The observation semantics ω_q defines how raw telemetry becomes an observed value $Y_q(a)$. The composition operator κ_q defines how components combine: by summation, maximum, interval union, critical path, retry expansion, or another rule.

Several examples illustrate why all fields are needed. For CPU time, the role is resource demand, the unit is seconds, the attribution boundary identifies CPU work attributed to the attempt, and composition is usually summation. For memory peak, the role is resource demand, the unit is bytes, but composition is a maximum rather than a sum. For worker-slot time, the role is worker occupancy, the unit is slot-seconds, the attribution boundary is the interval during which the attempt holds a worker slot, and composition is interval-based. For connection hold time, the role is worker occupancy, the unit is connection-seconds, and observation requires acquire and release events for the connection.

The interval expression for occupancy introduced above is one possible semantics encoded by κ_q. More generally, κ_q specifies how a dimension reduces component-level observations or estimates to an attempt-level value. For additive resource work, κ_q may be summation. For peak resources, it may be a maximum. For occupancy, it may be interval union with a capacity-time integral. For retry-sensitive work, it may expand over retry phases or repeated dependency calls.

For a task kind k, the relevant cost dimensions are

$$\mathcal{Q}_k = \mathcal{R}_k \dot{\cup} \mathcal{O}_k \subseteq \mathcal{Q}.$$

The subset $\mathcal{Q}_k$ is part of the task-kind model. A compute transform may include CPU time, allocation volume, and memory peak. A replication task may include network output bytes, disk read bytes, disk write bytes, worker-slot time, connection hold time, and lease time. The model therefore remains open-world: new domains can add dimensions without requiring one global list of all possible metrics.

5.5 Dimension Estimate Objects

A dimension estimator returns an estimate object rather than only one number. For a cost dimension q, let $\mathcal{E}_q$ be the estimate-object space. The estimate object for attempt a is

$$\widehat{E}_q(a) \in \mathcal{E}_q.$$

This object may contain a point estimate, quantiles, distributional summaries, conservative envelopes, missingness flags, and other information needed by diagnostic or policy layers.

The point estimate is a projection:

$$\widehat{Y}_q(a) = \mathrm{point}_q\Big(\widehat{E}_q(a)\Big), \qquad \widehat{Y}_q(a) \in \mathcal{V}_q.$$

The projection point_q is part of the dimension semantics or estimator design. It is not necessarily a mean. For some dimensions it may be an expected value; for others it may be a median, a conservative point, or a value selected for compatibility with a downstream controller.

When the estimate object contains distributional information, a quantile projection may also be defined:

$$\widehat{Y}_{q,p}(a) = \mathrm{quantile}_{q,p}\Big(\widehat{E}_q(a)\Big), \qquad p \in (0,1).$$

For example, an estimate object for worker-slot time may contain a median of two seconds, a 95th percentile of fifteen seconds, and a 99th percentile of two minutes. A throughput-oriented policy may inspect a point estimate, while a capacity-protection policy may inspect a high quantile. Both values come from the same estimate object.

The estimate-object abstraction is needed because queue-worker behavior is often tail-sensitive. If the model kept only one number per dimension, it would lose the information needed to distinguish stable short attempts from mostly short attempts with rare long occupancy tails. The later diagnostic and policy layers can use the same estimate object without changing the definition of cost.

5.6 Dimension-Specific Estimators

The estimator for a task kind is built from dimension-specific estimators. For each task kind k and dimension $q \in \mathcal{Q}_k$, define

$$F_{k,q} : \mathcal{X}_k \times \mathcal{C}_k \times \Theta_k \to \mathcal{E}_q.$$

This signature states that the estimator for dimension q uses intrinsic features, execution-context features, and reusable task-kind state, and returns an estimate object for that dimension. It does not require every dimension to use the same formula internally.

For attempt a, the dimension estimate is

$$\widehat{E}_q(a) = F_{k_a,q}(\mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}) .$$

The full task-kind estimator is the collection of these dimension-specific estimators:

$$F_k = \{F_{k,q} \mid q \in \mathcal{Q}_k\} .$$

Thus, evaluating the cost model for an attempt means selecting the relevant task kind, projecting its input features, reading the task-kind model state, and evaluating the estimators for the dimensions in $\mathcal{Q}_{k_a}$.

Some estimators may be direct rules or learned predictors. Others can be described through a component model. A common component model separates intrinsic baseline, contextual amplification, and additive overhead:

$$\widehat{E}_q(a) = \text{estimate}_q\Big(\widehat{B}_q(k_a, \mathbf{x}_a; \theta_{k_a}), \widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}), \widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}), \kappa_q\Big) .$$

This decomposition is a modeling pattern, not a required estimator interface. A descriptor or estimator may omit, fix, or replace any component when that component is not meaningful for the dimension.

The baseline $\widehat{B}_q$ estimates the contribution implied by the task kind and intrinsic features under a reference context. A reference context is a nominal or default execution context against which contextual amplification is measured. The amplification factor $\widehat{A}_q$ estimates how execution context modifies the baseline contribution. The overhead term $\widehat{O}_q$ estimates additional contribution that is not well represented as proportional to the baseline. The operator κ_q determines how these components are composed for dimension q.

For dimensions whose descriptor permits proportional amplification plus additive contribution, the point projection may use the affine specialization

$$\widehat{Y}_q(a) = \widehat{B}_q(k_a, \mathbf{x}_a; \theta_{k_a}) \cdot \widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}) + \widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}).$$

In this form, $\widehat{B}_q$ and $\widehat{O}_q$ have unit u_q, while $\widehat{A}_q$ is dimensionless. Values of $\widehat{A}_q$ above one represent amplification, values below one represent suppression, and value one represents no modeled contextual scaling. A negative amplification is not used to represent suppression; suppression is represented by a non-negative factor below one.

The affine form is not a universal law. It is appropriate only when the dimension supports proportional scaling and additive contribution. CPU time or some transfer-duration dimensions may fit this form. Memory peak may require a maximum-based operator. Worker-slot time may require interval or critical-path semantics. Retry-sensitive work may require retry expansion. A descriptor may therefore choose a non-affine estimator even when the same task kind uses affine estimators for other dimensions.

In the replication example, network output bytes may use payload size as a baseline feature. Peer health and network condition may amplify connection hold time or worker-slot time. Retry state may add overhead or expand the effective number of transfer attempts. These relationships should be encoded in the dimension estimator or in the Task Behavior Graph that parameterizes it.

5.7 Execution Overhead and Profiling Overhead

The term overhead needs a precise role. Overhead is not a third output family beside resource demand and worker occupancy. It is a contribution to one or more existing cost dimensions.

Execution overhead is overhead required by the task execution itself. Examples include setup, deserialization, connection establishment, lease-extension bookkeeping, acknowledgement, commit, and cleanup. Depending on the descriptor, these effects may contribute to CPU time, allocation volume, disk writes, worker-slot time, connection hold time, or other dimensions.

Profiling overhead is overhead introduced by observation policy. Examples include tracing, sampling, runtime profiling, high-cardinality label handling, telemetry export, and additional accounting. Profiling overhead can also contribute to cost dimensions, but only when the corresponding profiling policy is enabled.

This distinction motivates two related estimates. A base estimate excludes additional profiling overhead:

$$\widehat{\mathbf{C}}_{\text{base}}(a).$$

A policy-aware estimate includes the overhead of an already selected profiling policy ρ:

$$\widehat{\mathbf{C}}_{\rho}(a).$$

The symbol ρ denotes a profiling policy, while ρ_q denotes the role of a cost dimension. The subscript distinguishes the two uses. The cost model can represent both estimates, but it does not choose ρ. Policy selection is defined later. This separation prevents instrumentation from being treated as free while keeping the cost model independent of the policy decision.

5.8 Observed Values and Residuals

The model distinguishes estimates from observed values. For $q \in \mathcal{Q}_{k_a}$, the observed value $Y_q(a)$ is not raw telemetry. It is an attempt-attributed observed value produced according to the descriptor's observation semantics ω_q and attribution boundary α_q.

For example, node-level CPU utilization is not observed CPU usage for a particular attempt unless attribution is available. Worker reserve and release events can produce observed worker-slot time when they delimit the interval during which the slot is held. Connection acquire and release events can produce observed connection hold time. Wall-clock duration may provide useful evidence, but it is not worker-slot time unless the worker slot is held for the whole interval.

When an observed value exists, the residual is

$$\epsilon_q(a) = Y_q(a) - \widehat{Y}_q(a).$$

A positive residual means the observed value exceeded the point estimate. A negative residual means the point estimate exceeded the observed value. The residual is undefined when $Y_q(a)$ is unavailable; absence of a residual does not imply accuracy.

Residuals are evidence about the relation between the model and the workload. Large or systematic residuals may indicate missing features, stale context, hidden dependency state, model misspecification, drift, measurement error, or a tail event. Residuals can update θ_{k_a} for future attempts, feed diagnostics, or trigger a more detailed profiling policy. They are not themselves cost dimensions.

5.9 Distributional Cost Estimates

Attempt-level cost can be variable even within one task kind and one feature bucket. A single mean-like value may hide behavior that matters operationally. For queue workers, high-tail worker-slot time can reduce effective worker-pool capacity; high-tail connection hold time can exhaust dependency pools; retry storms can produce rare but expensive attempts.

The estimate-object formulation allows each dimension to carry distributional information. The task-kind model state θ_k may include empirical histograms, quantile sketches, parametric distribution parameters, conservative envelopes, or prediction intervals. The estimate object $\widehat{E}_q(a)$ may then expose projections such as $\widehat{Y}_{q,0.50}(a)$, $\widehat{Y}_{q,0.95}(a)$, or $\widehat{Y}_{q,0.99}(a)$.

Distributional estimates are still cost estimates. They should not be confused with uncertainty or risk. A high 99th percentile is a statement about estimated cost distribution. Confidence describes how reliable that estimate is. Risk describes the consequence of underestimating or mishandling it. Later sections derive diagnostics and risk interpretations from cost estimates, residuals, missingness, drift, tail ratios, and consequence information.

5.10 Boundaries of the Cost Model

This section defines attempt-attributed cost: active resource demand and worker occupancy. It also defines the objects needed to estimate and compare those quantities: feature projections, model state, typed dimensions, estimate objects, estimators, observed values, residuals, and distributional projections.

The section does not define the complete role system of the paper. Execution context, symptoms, outcomes, diagnostics, risk, and policy triggers are not cost outputs, even though they may influence or consume cost estimates. The next section defines those non-cost roles as influence domains. Task Behavior Graphs then use influence domains to connect canonical features, cost estimates, diagnostics, risk, and profiling-policy triggers in a task-kind-specific model.

6 Influence Domains

The cost model in Section 5 defines attempt-attributed cost: active resource demand and worker occupancy. A complete task-behavior model, however, also contains variables that are not cost outputs: task identity, intrinsic work, execution context, symptoms, outcomes, diagnostics, risks, and profiling-policy triggers. These variables must not be collapsed into one metric space, because they play different roles in estimation, diagnosis, risk interpretation, and profiling-policy selection.

Influence domains assign semantic roles to model variables. They are not telemetry sources and not metric categories. A metric, span event, log entry, queue event, profile sample, or domain event becomes part of the model only after it is mapped to a model variable with an assigned influence domain. The same raw observation may support more than one model variable, but each graph node has one primary role.

6.1 Domain Set

Let

$$\mathcal{D} = \{D_{\text{id}}, D_{\text{intr}}, D_{\text{ctx}}, D_{\text{res}}, D_{\text{occ}}, D_{\text{sym}}, D_{\text{out}}, D_{\text{diag}}, D_{\text{risk}}, D_{\text{policy}}\}$$

be the set of influence domains. The domains represent task identity, intrinsic features, execution context, resource demand, worker occupancy, symptoms, outcomes, reliability diagnostics, operational risk, and profiling-policy triggers.

The domains classify roles rather than data sources. For example, a queue event may be mapped to an outcome node, a worker lifecycle event may be mapped to an occupancy node, and a log entry may be mapped to a symptom node. The mapping depends on how the resulting variable is used in the task-behavior model.

6.2 Domain Assignment

For a Task Behavior Graph associated with task kind k, let V_k be the node set. The domain assignment is a typing function

$$\delta_k : V_k \to \mathcal{D}.$$

When the task kind is clear from context, we write $\delta(v)$ instead of $\delta_k(v)$.

This function assigns one primary influence domain to each graph node. If a raw signal supports several roles, the mapping layer should create several model variables rather than assigning several roles to one node. For example, a dependency-timeout log entry may support a symptom variable, an attempt-outcome variable, and risk evidence, but those roles should be represented by distinct nodes:

$$\delta(\texttt{dependency\_timeout\_symptom}) = D_{\text{sym}},$$

$$\delta(\texttt{attempt\_timeout\_outcome}) = D_{\text{out}},$$

$$\delta(\texttt{retry\_storm\_risk}) = D_{\text{risk}}.$$

This one-domain rule makes graph interpretation and validation explicit.

6.3 Domain Semantics

Table 3 summarizes the domains, their roles, typical examples, and common confusions. The table is part of the model definition: it specifies how variables may be interpreted before they are connected by graph edges.

Table 3: Influence domains and their semantic roles.

Domain	Role	Examples	Common confusion
D_{id}	Identifies which task is being modeled and selects reusable task-kind structure.	task kind, task version, queue, tenant, producer, priority, deadline	Identity selects a model; it is not a cost value.
D_{intr}	Describes work contained in the task instance.	payload size, record count, object count, batch size, partition size, operation mode	Intrinsic work is not execution context.

Domain	Role	Examples	Common confusion
D_{ctx}	Describes the environment and state under which the attempt executes.	dependency health, network condition, disk pressure, cache state, worker-pool pressure, retry state	Context may influence cost, but it is not an attempt-attributed cost output.
D_{res}	Describes active resource demand or observed active resource usage attributed to the attempt.	CPU time, memory peak, allocation volume, disk I/O, network I/O, accelerator time	Resource demand is not bounded-capacity occupancy.
D_{occ}	Describes bounded execution capacity held by the attempt over time.	worker-slot time, lease time, connection hold time, lock hold time, semaphore-permit time	Occupancy is not wall-clock duration by itself.
D_{sym}	Describes an intermediate observed signal that indicates relevant or abnormal behavior.	dependency wait spike, retry burst, lock-contention signal, queue-delay spike	A symptom is not necessarily the root cause or final outcome.
D_{out}	Describes how the attempt completed.	success, failure, timeout, cancellation, retryable error, dead-lettering, lease expiration	An outcome is not a cost value by itself.
D_{diag}	Describes reliability of estimates or model fit.	residual magnitude, missingness, low sample count, drift, tail ratio, multimodality, confidence	A diagnostic is not an operational consequence by itself.
D_{risk}	Describes consequence-weighted operational danger.	queue starvation, retry storm, SLO violation, replication lag, tenant impact, cascading overload	Risk is not active resource demand or worker occupancy.
D_{policy}	Describes a profiling-policy trigger or observation response.	minimal accounting, detailed accounting, tracing, tail-aware profiling, diagnostic profiling	A profiling-policy trigger is not a scheduling or placement decision.

6.4 Permitted Domain Flow

The domain assignment constrains graph edges. Let

$$\rightsquigarrow_{\mathcal{D}} \subseteq \mathcal{D} \times \mathcal{D}$$

denote the permitted domain-flow relation. If $(u, v) \in E_k$, then the edge is well typed only when

$$\delta_k(u) \rightsquigarrow_{\mathcal{D}} \delta_k(v).$$

This condition does not prove causality. It only states that a graph edge uses the source variable in an admissible role relative to the target variable.

Typical permitted flows are:

$$D_{\text{id}} \rightsquigarrow_{\mathcal{D}} D_{\text{intr}}, D_{\text{ctx}}, D_{\text{res}}, D_{\text{occ}}, D_{\text{policy}},$$

because identity can select task-kind structure, relevant features, relevant cost dimensions, and policy priors.

Intrinsic features and execution context may influence cost estimates:

$$D_{\text{intr}},\ D_{\text{ctx}} \rightsquigarrow_{\mathcal{D}} D_{\text{res}},\ D_{\text{occ}}.$$

For example, payload size may influence network output bytes, while dependency health may influence worker-slot time or connection hold time.

Context may also influence symptoms and outcomes:

$$D_{\text{ctx}} \rightsquigarrow_{\mathcal{D}} D_{\text{sym}},\ D_{\text{out}}.$$

For example, degraded dependency health may be associated with dependency-wait symptoms or timeout outcomes.

Cost, symptoms, and outcomes may feed diagnostics or risk interpretation:

$$D_{\text{res}},\ D_{\text{occ}},\ D_{\text{sym}},\ D_{\text{out}} \rightsquigarrow_{\mathcal{D}} D_{\text{diag}},\ D_{\text{risk}}.$$

For example, a large worker-slot residual may feed reliability diagnostics, and a timeout outcome may feed retry-storm risk.

Diagnostics and risk may feed profiling-policy triggers:

$$D_{\text{diag}},\ D_{\text{risk}} \rightsquigarrow_{\mathcal{D}} D_{\text{policy}}.$$

For example, low confidence together with high queue-starvation risk may trigger tail-aware profiling.

Some flows are restricted. Outcomes and diagnostics for attempt a may update task-kind state for future attempts, but they must not be used as pre-execution input features for the same attempt. Otherwise, the graph would leak post-execution information into the estimate it is supposed to produce. In particular, observed values, residuals, and final outcomes for attempt a must not be folded back into z_a for the same pre-execution estimate.

6.5 Ambiguous Signals

Some operational signals are ambiguous until the model assigns a role. Duration, retry count, timeout, and queue delay are common examples.

A duration signal may represent different roles. Attributed CPU duration belongs to D_{res}. Worker-slot time belongs to D_{occ}. Dependency wait time is often a symptom in D_{sym}, unless a descriptor explicitly defines it as an occupancy dimension. Elapsed wall-clock duration is usually an outcome-related observation, not worker occupancy by itself.

A retry-count signal may also have several roles. The attempt number available before execution may be execution context. A burst of retries observed during execution may be a symptom. A retry scheduled after failure may be an outcome. A high retry count may also be evidence for retry-storm risk. These roles should not be represented by one overloaded graph node.

This distinction is important because the same raw observation can enter the model through several typed variables:

$$s \mapsto \{v_1, \ldots, v_m\}, \qquad \delta_k(v_j) \in \mathcal{D}.$$

The mapping layer may derive several variables from one raw signal, but the graph layer keeps their roles separate.

6.6 Anti-Double-Counting Rule

The anti-double-counting rule is central: variables that describe the same incident must not be aggregated as if they were independent additive costs. Network instability, dependency wait time, retry count, wall-clock duration, connection hold time, and worker-slot time may all appear during one degraded attempt. They do not have the same role.

Consider an attempt that runs during packet loss and waits thirty seconds on a degraded dependency. The incident may produce the following variables:

$$\delta(\texttt{network\_condition}) = D_{\text{ctx}},$$

$$\delta(\texttt{dependency\_wait\_spike}) = D_{\text{sym}},$$

$$\delta(\texttt{retry\_count}) \in \{D_{\text{ctx}}, D_{\text{sym}}, D_{\text{out}}\},$$

depending on whether retry count is used as pre-execution attempt state, an observed retry burst, or a completion result,

$$\delta(\texttt{wall\_clock\_duration}) = D_{\text{out}},$$

$$\delta(\texttt{worker\_slot\_time}) = D_{\text{occ}},$$

and

$$\delta(\texttt{queue\_starvation\_risk}) = D_{\text{risk}}.$$

Only some of these variables are cost dimensions. Network condition is context. Dependency wait is a symptom unless it is explicitly modeled as a held-capacity dimension. Wall-clock duration is an elapsed outcome. Worker-slot time is occupancy only for intervals during which the worker slot is held. Queue starvation is a consequence, not an additional resource-demand dimension.

The model must assign roles before aggregation. This prevents a single underlying incident from being counted once as context, again as a symptom, again as duration, again as occupancy, and again as risk. Role assignment does not remove the relationships between these variables; it makes those relationships explicit.

6.7 Boundaries

Influence domains define how model variables may be interpreted and connected. They do not define the cost estimator, do not map raw telemetry into canonical features, do not compute uncertainty or risk, do not select the profiling policy, and do not prove causality. Those responsibilities belong to later sections.

The role of this section is narrower: it provides the typing discipline used by Task Behavior Graphs. Section 8 uses the domain assignment to connect typed variables into task-kind-specific graphs. Section 9 explains how raw observations are mapped into model variables. Sections 12 and 13 consume cost estimates, diagnostics, and risk to reason about reliability and observation policy.

7 Task Taxonomy

Task kinds differ in the features that explain their behavior, the cost dimensions they affect, and the contexts that amplify their costs. This section introduces a working taxonomy used by later examples. The taxonomy is not a complete classification of all workloads.

Compute-transform tasks are shaped by record count, payload size, algorithm mode, CPU pressure, allocation behavior, and memory pressure. Their main cost dimensions are CPU time, allocation volume, and memory peak.

I/O transfer tasks are shaped by byte count, object count, chunk size, disk pressure, network condition, and peer health. Their main cost dimensions are disk reads, disk writes, network input, network output, worker-slot time, and connection hold time.

External-dependency tasks are shaped by dependency health, fan-out width, timeout policy, connection-pool pressure, and retry state. Their active CPU demand may be low while worker-slot time and connection hold time are high.

Replication and repair tasks are shaped by payload size, replica count, divergence level, peer health, replication lag, disk pressure, and retry state. They often combine I/O transfer, coordination, and external-dependency behavior.

Coordination-sensitive tasks are shaped by locks, leases, semaphores, tenant concurrency, and contention. Their active resource demand may be low while occupancy dominates system impact.

A single task kind may combine several classes. Task Behavior Graphs represent this compositional behavior through typed nodes and edges rather than by forcing each task kind into one exclusive class.

8 Task Behavior Graphs

The cost model defines typed dimensions and estimate objects. Influence domains define semantic roles and permitted role flow. A Task Behavior Graph connects these roles for a task kind or task family.

For task kind k, a Task Behavior Graph is a typed directed graph

$$\mathcal{G}_k = (V_k, E_k, \delta_k, \tau, \lambda, w, \gamma)\,.$$

Here V_k is the set of nodes, $E_k \subseteq V_k \times V_k$ is the set of directed influence edges, $\delta_k : V_k \to \mathcal{D}$ assigns an influence domain to each node, τ assigns a node type, λ assigns an edge role, w stores edge parameters, and γ stores confidence values.

Nodes represent typed model variables rather than raw metric names. A node may represent task identity, an intrinsic feature, an execution-context feature, a resource-demand estimate object, a worker-occupancy estimate object, a symptom, an outcome, a reliability diagnostic, an operational-risk output, or a profiling-policy trigger. Each node has one primary influence domain; if one raw observation supports several roles, the mapping layer creates several variables before they appear in the graph.

Edges represent modeled influence assumptions and must respect the permitted domain-flow relation from Section 6.4. Edge roles include baseline influence, contextual amplification, overhead contribution, evidence link, symptom link, outcome link, diagnostic link, risk propagation, and policy trigger:

$$\lambda(e) \in \{\text{baseline}, \text{amplification}, \text{overhead}, \text{evidence}, \text{symptom}, \text{outcome}, \text{diagnostic}, \text{risk}, \text{policy}\}.$$

An edge is not proof of causality. It records an explicit assumption used by the task-kind model. The confidence value γ_e records how reliable that assumption is considered by the current model state or manifest.

A Task Behavior Graph may implement a dimension-specific estimator $F_{k,q}$, parameterize it, or document the influence structure used by a separate estimator. The required property is that the graph makes task-kind-specific influence assumptions explicit while preserving the role boundaries defined by influence domains. In particular, outcome, diagnostic, and risk nodes must not be used as pre-execution intrinsic or context inputs for the same attempt.

A replication graph fragment illustrates the structure. The feature `payload_size_bytes` may be an intrinsic node that influences `network_output_bytes` through a baseline edge. The context feature `peer_health` may influence `worker_slot_time` through an amplification edge. The

context feature `retry_attempt` may influence `connection_hold_time` and a retry diagnostic. The cost node `worker_slot_time` may feed a queue-starvation risk node, and that risk node may trigger a tail-aware profiling policy.

9 Metrics, Signals, and Features

Raw observations are not model variables. Metrics, spans, logs, runtime profiles, queue events, worker lifecycle events, and domain events become useful only after they are mapped into typed model variables with assigned influence domains.

Let s be a raw signal from an observation source. A mapping rule is a typed transformation

$$M_j : s \mapsto m_j,$$

where m_j is a derived model variable and $\delta(m_j) \in \mathcal{D}$ is its influence-domain assignment. The target may be a canonical input feature, an observed cost value, a symptom, an outcome, diagnostic evidence, risk evidence, or a profiling-policy trigger.

The mapping target matters. A packet-loss metric may become a network-condition context feature. Worker reserve and release events may become observed worker-slot occupancy. A dependency timeout log may become a symptom if it is used as intermediate evidence, or an outcome if it is the terminal result of an attempt. A sustained queue delay may become risk evidence when it describes the consequence of occupancy or retries.

A single raw observation may map to several variables. For example, a dependency timeout record may produce a symptom node, an outcome node, and risk evidence. The one-domain rule still applies: each derived variable receives one primary domain, and the relations among those variables are represented by graph edges. This avoids treating one observation as several independent costs.

Canonical features form the input vector z_a. Observed output values such as $Y_q(a)$ are not part of z_a for the same pre-execution estimate; they are produced during or after execution and used for residuals, calibration, diagnostics, and future model updates. This separation prevents post-execution leakage while still allowing the model to learn from historical attempts.

10 Profiling Methods

Profiling methods differ in precision, overhead, and attribution quality. Minimal accounting records task lifecycle events and basic outcomes. Detailed accounting records typed cost dimensions such as worker-slot intervals, connection-hold intervals, and byte counts. Tracing records causal structure across services and dependencies. Runtime profiling samples CPU, allocation, or blocking behavior. Diagnostic profiling enables more expensive observation for uncertain or high-risk task classes.

No method is universally best. The profiling policy should select the lowest overhead method that provides enough evidence for the current uncertainty and risk level.

11 Observation Points

A queue-worker lifecycle exposes several observation points: enqueue, reserve, start, dependency call, retry, backoff, commit, acknowledge, fail, release, and cleanup. Each point can produce raw observations, but the model uses them only after role assignment. Reserve and release events may support worker-slot occupancy. Dependency acquire and release events may support connection-hold occupancy. Retry events may support symptoms, diagnostics, or context features.

12 Uncertainty, Confidence, and Risk

This section treats uncertainty as the primary concept. Entropy is optional and only meaningful when a discrete predictive distribution is defined.

Reliability diagnostics describe the trustworthiness of the current behavior estimate. They are downstream from cost estimates and observations, but they are not cost dimensions. Let

$$\mathrm{Diag}(a) = h_{\mathrm{diag}}\Big(\widehat{\mathbf{C}}(a), \{\epsilon_q(a)\}, z_a, \theta_{k_a}\Big)$$

denote a structured diagnostic object when the required inputs are available. Diagnostics may include residual magnitude, missingness, sample scarcity, drift, tail ratio, multimodality, calibration error, and estimator confidence. These diagnostics belong to D_{diag} in the influence-domain system.

Risk is downstream from cost and diagnostics:

$$\mathrm{Risk}(a) = g_{\mathrm{risk}}\Big(\widehat{\mathbf{C}}(a), \mathrm{Diag}(a), S(a); \theta_{k_a}\Big),$$

where $S(a)$ denotes consequence information such as tenant criticality, dependency fragility, queue-starvation impact, or SLO sensitivity. Risk belongs to D_{risk}. It is not active resource demand, not worker occupancy, and not merely low confidence. Low confidence can increase risk only when the consequence of being wrong is operationally significant.

The separation between diagnostics and risk prevents a common modeling error. A large residual says that the model and the observed workload disagreed. It does not by itself say that the attempt is dangerous. A high-risk interpretation requires consequence information: for example, whether underestimating this attempt can starve a queue, overload a dependency, violate an SLO, or amplify a retry storm.

13 Profiling Policy Selection

Profiling policy selection decides how much observation is justified. It consumes cost estimates, reliability diagnostics, operational-risk interpretation, and an observation budget. It does not decide final scheduling, routing, placement, or admission.

Let $\mathcal{P}$ be a set of available profiling policies and let B_{obs} be the observation budget. A selector may be written as

$$\rho(a) = \arg\operatorname*{select}_{p \in \mathcal{P}} J_p\Big(\widehat{\mathbf{C}}(a), \mathrm{Diag}(a), \mathrm{Risk}(a), B_{\mathrm{obs}}\Big),$$

where J_p denotes a policy score or rule predicate. The notation arg select is intentionally more general than an optimization operator: a production system may implement policy selection as a ranked rule set, a threshold rule, a constrained optimizer, or a controller.

Profiling policies are represented in the influence-domain model as D_{policy} outputs. A stable low-risk task may use minimal accounting. A task with high-tail occupancy, systematic residuals, missing context, or high consequence may justify detailed accounting, sampled tracing, runtime profiling, or diagnostic profiling. The selected policy may affect a policy-aware cost estimate through profiling overhead, but it must not alter the base cost estimate for the same attempt.

14 Task Behavior Manifests

Task Behavior Manifests provide a declarative bridge between the model and implementation. A manifest declares task classes or task kinds, canonical features, cost dimensions, observation mappings, graph fragments, diagnostics, risk rules, and profiling policies.

The manifest layer uses the terminology defined in the previous sections. A feature declaration must identify whether the variable is intrinsic work or execution context. A cost-dimension declaration must provide or reference a descriptor Δ_q. A graph node must have one influence-domain assignment. A graph edge must respect the permitted domain-flow relation unless the manifest explicitly marks the edge as implementation-specific and outside the base behavior model.

A replication manifest may declare `payload_size_bytes` as an intrinsic feature, `peer_health`, `network_condition`, and `retry_attempt` as context features, `network_output_bytes` and `disk_read_bytes` as resource-demand dimensions, and `worker_slot_time` and `connection_hold_time` as worker-occupancy dimensions. It may then declare graph edges from payload size to network output, from peer health to worker-slot time, from retry state to connection-hold time, from residual diagnostics to retry-storm risk, and from risk to a tail-aware profiling policy.

Manifests allow domain packs to extend the model without requiring the core system to know all possible domain metrics in advance. The core requirement is not that every metric be predefined, but that every introduced variable has a role, every cost dimension has a descriptor, and every graph edge has a semantically valid interpretation.

15 Case Studies

This section outlines worked examples used to validate the model structure. A replication case study should show how payload size, peer health, network condition, and retry state influence network output, connection-hold time, and worker-slot time. A compute-transform case study should contrast high active CPU demand with lower occupancy sensitivity. An external-dependency case study should show low CPU demand but high occupancy under degraded dependencies.

The current paper treats these as structured examples rather than empirical production results.

16 Evaluation Methodology

The model can be evaluated along four axes: attribution quality, estimate accuracy, diagnostic usefulness, and profiling overhead. Attribution quality checks whether observed values are correctly assigned to attempts. Estimate accuracy compares point and quantile estimates with observed values. Diagnostic usefulness checks whether residuals, missingness, drift, and tail indicators explain model failures. Profiling overhead measures the cost of additional instrumentation.

17 Discussion

The model is intentionally modular. Cost dimensions, graph nodes, observation mappings, diagnostics, and policies can evolve independently when their interfaces remain typed. This structure supports domain-specific extension, but it also introduces maintenance cost. A deployment must decide which task kinds justify detailed modeling and which can remain under coarse accounting.

18 Threats to Validity

The paper presents a modeling framework rather than a completed empirical evaluation. The main threats are model misspecification, incomplete instrumentation, weak attribution, domain-

specific hidden state, stale historical data, and profiling overhead. Task Behavior Graphs expose influence assumptions, but they do not prove causality without additional evidence.

19 Related Work

This work relates to cluster resource management, workload characterization, tail-latency analysis, distributed tracing, telemetry systems, workload classification, and queue-based execution control. Prior work provides multi-resource allocation models, large-scale workload observations, and observability mechanisms. This paper focuses on the missing task-attempt layer: typed cost attribution, role assignment for observations, and profiling-policy selection for queue workers.

20 Conclusion

This paper introduced Task Behavior Graphs as a domain-driven model for profiling queue-worker task attempts under uncertain resource demand. The model separates task identity, intrinsic features, execution context, resource demand, worker occupancy, observations, diagnostics, risk, and profiling policy. The central contribution is a formal structure for representing task-kind-specific behavior and for deciding how much observation is justified. Future work should implement the manifest layer, validate the model on real queue workloads, and measure the overhead of policy-driven profiling.

References

[1] Yue Cheng, Ali Anwar, and Xuejing Duan. Analyzing alibaba's co-located datacenter workloads. In *2018 IEEE International Conference on Big Data*, Big Data '18. IEEE, 2018.

[2] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[3] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, OSDI '04, pages 137–150. USENIX Association, 2004.

[4] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13. ACM, 2013.

[5] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14. ACM, 2014.

[6] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '11. USENIX Association, 2011.

[7] OpenTelemetry. Opentelemetry documentation: Sampling. `https://opentelemetry.io/docs/concepts/sampling/`, 2025. Accessed 2026-05-03.

[8] OpenTelemetry. Opentelemetry documentation: Performance. `https://opentelemetry.io/docs/zero-code/java/agent/performance/`, 2026. Accessed 2026-05-03.

[9] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing*, SoCC '12. ACM, 2012.

[10] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.

[11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15. ACM, 2015.

[12] Qi Zhang, Joseph L. Hellerstein, and Raouf Boutaba. Characterizing task usage shapes in google compute clusters. In *Proceedings of the 5th International Workshop on Large Scale Distributed Systems and Middleware*, LADIS '11. ACM, 2011.

A Extended Terminology

This appendix provides the complete terminology reference used by the paper. Section 2 intentionally keeps only the core terms needed for the main model. The extended glossary is organized by semantic area so that the main text can introduce concepts progressively while still keeping a precise reference for task entities, behavior classes, execution context, observations, influence domains, resource demand, worker occupancy, uncertainty, risk, graph structure, profiling policy, manifests, and queue-worker lifecycle stages.

A.1 Task Entities

Table 4: Task entity terms.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue, such as copying a replication partition, rebuilding a cache entry, resizing an image, repairing a database record, dispatching a webhook, or validating a document. A task kind describes what work is requested, but it does not fully determine how that work behaves under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution attempt of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource demand, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.

Term	Definition
task family	A group of task kinds that share a similar behavior-graph structure or profiling model. A task family can be used when one graph template applies to several related task kinds.
task identity	The identifying metadata of a task, including task kind, task version, queue, producer, tenant, priority, and deadline. Task identity identifies the work but does not itself define the cost of executing it.
task version	The version of a task kind, handler contract, payload schema, or behavior manifest. Task versions are useful for compatibility between historical profiles, manifests, and schema versions.
queue	The queue through which a task instance is submitted, stored, reserved, and eventually consumed by a worker. A queue name may provide useful context, but it is not a complete behavior model.
producer	The component that creates and submits a task instance to a queue. The producer may provide metadata or declared hints, but it does not fully determine the execution context.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
worker pool	A set of workers that execute task attempts. Worker-pool capacity and worker-pool occupancy are central to queue-worker profiling.
tenant	A user, customer, namespace, application, or logical group that consumes execution capacity. Tenants are relevant for isolation, fairness, consequence analysis, and operational risk.
tenant class	A category of tenants based on SLO, priority, plan, criticality, or operational risk. A tenant class is distinct from a task kind.
priority	A scheduling or ordering attribute assigned to a task instance. Priority may influence profiling or admission decisions, but it is not itself resource demand or operational risk.
deadline	A completion target or time constraint associated with a task instance. A deadline may affect consequence and risk, but it is distinct from observed latency.

A.2 Task Behavior

Table 5: Task behavior terms.

Term	Definition
task behavior	The observed or estimated execution pattern of a task kind, task instance, or task attempt under a particular execution context. Task behavior includes resource demand, worker occupancy, retry behavior, dependency wait, outcome, uncertainty, and risk-relevant effects.
behavior profile	A summarized description of how a task kind behaves under a class of execution contexts. A behavior profile may include demand distributions, occupancy distributions, dominant behavior classes, uncertainty, risk class, and relevant features.
behavior class	An analytical category derived from observed or estimated behavior. Examples include compute-dominant, dependency-bound, retry-amplifying, worker-slot-heavy, payload-sensitive, state-dependent, or high-tail-risk behavior.
behavior mode	A context-dependent mode of behavior for a task kind. The term emphasizes that the same task kind may behave differently under different execution contexts.
compute-dominant behavior	Behavior in which CPU demand is the dominant active resource dimension. A task kind may exhibit compute-dominant behavior without being intrinsically a CPU task in all contexts.
memory-pressure-sensitive behavior	Behavior that is sensitive to memory pressure, allocation rate, working-set size, or garbage-collection effects. This behavior is not necessarily identical to being memory-bound.
disk-IO-dominant behavior	Behavior in which disk reads, disk writes, storage throughput, or I/O wait dominates execution. Disk I/O demand is distinct from disk pressure in the execution environment.
network-sensitive behavior	Behavior that changes substantially with network condition, such as packet loss, jitter, retransmissions, bandwidth changes, or timeouts.
dependency-bound behavior	Behavior constrained primarily by external dependencies, such as databases, APIs, remote services, rate limits, quotas, or connection pools.
retry-amplifying behavior	Behavior in which retries substantially increase work, worker occupancy, or operational risk. Retry count may be a symptom or outcome rather than an independent cause.

Term	Definition
worker-slot-heavy behavior	Behavior with high worker-slot time relative to active resource usage. This class is important for queue workers because a task may consume little CPU but hold scarce worker capacity for a long time.
payload-sensitive behavior	Behavior that depends strongly on payload size, input shape, record count, object count, or batch size. Payload-sensitive behavior can still be uncertain if context or hidden state varies.
state-dependent behavior	Behavior that depends on domain-specific or internal system state, such as replica divergence, cache state, compaction debt, segment fragmentation, or dirty data volume.
high-tail-risk behavior	Behavior in which rare slow executions or high-percentile outcomes are operationally important. High-tail-risk behavior is distinct from high average cost.
stable behavior	Behavior with low residual error, low drift, and low variability under comparable execution contexts. Stable behavior may still be expensive.
unstable behavior	Behavior with high variability, drift, residual error, multimodality, or context sensitivity. Unstable behavior is not necessarily high-cost in every attempt.

A.3 Execution Context

Table 6: Execution-context terms.

Term	Definition
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.
context factor	A feature of the execution context that may influence task behavior. Context factors are modeled as influences, not automatically as proven causes.
worker state	The local state of the worker during a task attempt, including local runtime pressure, running work, memory state, garbage-collection pressure, and local scheduling delay.
worker pressure	Pressure on a worker or worker pool, such as busy worker slots, saturated local queues, high utilization, or local scheduling delay.

Term	Definition
node pressure	Pressure on the host, virtual machine, container, or node on which the worker runs. Examples include CPU saturation, memory pressure, disk pressure, and network saturation.
dependency health	The operational condition of an external dependency, such as latency, timeout rate, error rate, saturation, quota pressure, or availability.
dependency capacity	The available capacity of an external dependency, such as connection-pool capacity, rate-limit budget, service quota, or downstream throughput.
network condition	The state of the network path relevant to a task attempt, including round-trip time, jitter, packet loss, retransmissions, bandwidth, timeouts, and availability.
network instability	A normalized feature representing unstable network behavior. It may be derived from packet loss, jitter, retransmissions, timeout rate, bandwidth variation, or recent network failures.
replica state	The state of a replica, shard, peer, or replica set involved in a task attempt. Replica state is relevant to replication, repair, synchronization, and consistency tasks.
replica health	A normalized feature representing the operational condition of a replica or peer. Replica health is related to, but distinct from, replication lag.
replication lag	An observation or signal describing how far a replica is behind another source of truth. It may be measured in bytes, records, operations, versions, or time.
disk pressure	Pressure on the storage subsystem, such as queue depth, read latency, write latency, I/O wait, or saturation. Disk pressure is an execution-context factor, not the same as disk I/O demand.
cache state	The condition of a cache relevant to a task attempt, such as warm, cold, stale, evicted, partially populated, or inconsistent.
cache warmth	A normalized feature representing how prepared or populated a cache is for the task attempt. Cache warmth can influence resource demand and latency.
data locality	The proximity of required data to the worker or execution environment. Examples include local, remote, cross-zone, cross-region, or external storage access.
tenant pressure	Pressure or quota usage associated with a tenant, such as used concurrency, rate-limit consumption, or tenant-level backlog.

Term	Definition
retry state	The state of the retry cycle for a task instance or attempt, including attempt number, previous errors, backoff state, retry budget, and retry history.
hidden system dynamics	Unobserved or partially observed system state that affects task behavior, such as internal dependency queues, lock contention, cache eviction history, data fragmentation, noisy neighbors, kernel behavior, or runtime effects. Hidden dynamics are represented through residual error, uncertainty, and reduced confidence when not captured by known features.

A.4 Observations, Signals, and Features

Table 7: Observation, signal, and feature terms.

Term	Definition
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.
raw observation	An unnormalized measurement or event before interpretation. A raw observation does not by itself have stable model meaning.
metric	A time-series, counter, gauge, or histogram observation. Metrics are one kind of observation; traces, logs, profiles, and domain events are observations too.
counter	A metric that counts events, operations, bytes, retries, failures, or other monotonic quantities. A counter may represent a symptom or outcome rather than a root influence.
gauge	A metric representing a current value, such as queue depth, active workers, available connections, or current memory usage.
histogram	A metric representing a distribution of observed values, such as durations, latencies, sizes, or per-attempt costs.
trace	An execution trace composed of spans. A trace exposes execution structure and timing, but it does not by itself define task-kind-specific relevance, causality, feature mapping, or profiling policy.
span	A timed unit inside a trace representing an operation, processing stage, dependency call, or instrumented region. A span is not the same as a task attempt.

Term	Definition
log	A structured or unstructured event record emitted by a system. Logs may expose status, error classes, domain events, or diagnostic information.
runtime profile	A CPU, heap, allocation, lock, blocking, or runtime-level profile. A runtime profile is distinct from a behavior profile.
signal	An interpreted, aggregated, or derived observation that indicates a condition relevant to the model. A signal is an intermediate layer between raw observation and canonical feature.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
semantic feature	An informal synonym for a feature with explicit semantic meaning. This paper uses the term canonical feature when referring to model inputs with defined type, range, unit, influence domain, and intended meaning.
feature mapping	The process of converting raw observations, signals, task metadata, and context into canonical features. Feature mapping is separate from graph evaluation.
normalization	The process of converting values into a defined type, unit, scale, bucket, or range. For example, network measurements may be normalized into an instability score in $[0, 1]$.
aggregation	The process of combining observations over a time window, group, task kind, attempt set, or distribution. Examples include sum, average, maximum, p95, and p99.
feature registry	A registry of canonical features, including names, types, ranges, units, influence domains, and intended meaning.
metric descriptor	A descriptor of a raw metric, including source, unit, label set, aggregation, cardinality, and mapping to canonical features.
feature descriptor	A descriptor of a canonical feature, including type, range, unit, semantic meaning, influence domain, and expected use in the model.
cardinality	The number of distinct time series or label combinations produced by a metric. Cardinality affects telemetry overhead and storage cost.
high-cardinality label	A metric label that can create a large or unbounded number of series, such as task id, user id, object id, URL, or unbounded tenant label.

Term	Definition
observation window	The time window over which observations are aggregated or interpreted. For example, network instability may be computed over the last 30 seconds.
freshness	How recent and applicable an observation is to the current task attempt. Stale observations may reduce confidence.

A.5 Influence Domains

Table 8: Influence domain terms.

Term	Definition
influence domain	A semantic role assigned to a model variable in the task behavior model. Influence domains are not metric domains and not telemetry sources: they group model elements by role rather than by where a signal was observed.
task identity domain	The influence domain containing identifiers and metadata that describe which task is being executed, such as task kind, version, queue, tenant, producer, priority, deadline, and attempt number.
intrinsic feature domain	The influence domain describing work contained in the task instance itself, such as payload size, record count, object count, partition size, input format, operation mode, or batch size.
execution context domain	The influence domain describing the input-side environment and state under which the task attempt executes, such as network condition, dependency health, replica state, disk pressure, cache warmth, worker pressure, retry state, and tenant pressure.
resource demand domain	The influence domain describing active resource dimensions required or consumed by the task attempt, such as CPU time, memory peak, disk I/O, network I/O, runtime allocations, or accelerator usage. Waiting on dependencies is not active resource demand by itself, although it can contribute to worker occupancy or risk.
worker occupancy domain	The influence domain describing bounded execution capacity held by the task attempt, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore-permit time, or tenant-concurrency time.

Term	Definition
symptom domain	The influence domain describing intermediate observed signals that indicate abnormal or relevant behavior but are not final outcomes and not cost dimensions by themselves. Examples include dependency wait spikes, retry bursts, timeout warnings, queue-delay spikes, and lock-contention signals.
outcome domain	The influence domain describing how the task attempt completed or transitioned in its lifecycle, such as success, failure, timeout, cancellation, retryable error, non-retryable error, dead-lettering, or lease expiration.
reliability diagnostic domain	The influence domain describing estimate reliability and model-quality evidence, such as residual magnitude, missing features, sample scarcity, drift, tail ratio, multimodality, calibration error, or confidence.
risk domain	The influence domain describing the consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Examples include queue starvation, retry storm, SLO violation, replication lag, tenant impact, dependency overload, and cascading overload.
profiling-policy trigger domain	The influence domain describing variables or decisions that select an observation strategy, such as minimal accounting, detailed accounting, sampled tracing, tail-aware profiling, or diagnostic profiling.
one-domain rule	The rule that each Task Behavior Graph node has one primary influence domain. A single raw observation may map to several model variables when it supports several roles.
anti-double-counting rule	The rule that a signal must be assigned to its model role before it is aggregated, scored, or used as evidence for another role.

A.6 Resource Demand, Occupancy, Cost, and Overhead

Table 9: Resource demand, occupancy, cost, and overhead terms.

Term	Definition
resource demand	The expected or required active resource need of a task attempt along a resource dimension. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.

Term	Definition
active resource usage	Actual use of active resources such as CPU, memory, disk, network, runtime allocation, or accelerator resources. Active resource usage is distinct from waiting and bounded-capacity occupancy.
CPU time	The amount of CPU execution time consumed by a task attempt. CPU time is not the same as wall-clock duration.
CPU pressure	Pressure on CPU capacity in the worker, container, node, or co-located environment. CPU pressure is an execution-context factor, not the same as task CPU demand.
memory peak	The maximum memory used by a task attempt during execution. Memory peak is a resource-usage or resource-demand dimension.
memory pressure	Pressure on available memory in the worker, container, node, or runtime environment. Memory pressure is an execution-context factor.
allocation behavior	The allocation volume, allocation rate, allocation pattern, or allocation-induced runtime effect of a task attempt. Allocation behavior may influence memory pressure, garbage collection, and CPU time.
disk I/O	Disk reads, writes, operations, bytes, or I/O wait associated with a task attempt. Disk I/O demand is distinct from disk pressure.
network I/O	Network bytes, packets, requests, or transfer volume associated with a task attempt. Network I/O is distinct from network instability.
wait time	Time spent waiting rather than actively consuming a hardware or runtime resource. Wait time may be caused by dependencies, locks, network transfer, backoff, throttling, local scheduling delay, or other blocking conditions.
dependency wait time	Time spent waiting for an external dependency, such as a database, API, remote service, or storage system. Dependency wait time is not active resource demand by itself, but it can increase wall-clock duration and worker occupancy when bounded worker capacity remains held during the wait.
worker occupancy	Bounded execution capacity held by a task attempt over time. Worker occupancy is a first-class cost component in queue-worker systems.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension.
lease time	The time during which a task lease or reservation is held. Lease time may differ from worker-slot time.

Term	Definition
connection hold time	The time during which a task attempt holds a connection to a dependency, database, broker, or external service.
lock hold time	The time during which a task attempt holds a lock. Lock hold time can affect other tasks and may contribute to contention.
lock wait time	The time spent waiting to acquire a lock. Lock wait time is distinct from lock hold time.
semaphore hold time	The time during which a task attempt holds a semaphore, concurrency token, or bounded execution permit.
tenant concurrency time	The time during which a task attempt consumes a tenant-level concurrency slot. This dimension is relevant to fairness and isolation.
wall-clock duration	Elapsed time from attempt start to attempt finish. Wall-clock duration is often related to worker-slot time, but the two are not necessarily identical.
latency	Delay observed by a caller, queue, worker, dependency, or system boundary. Latency is an outcome or symptom, not resource demand by itself.
service time	The time a task attempt spends being served or executed, excluding queue wait time. The exact boundary must be defined for the queue-worker system being analyzed.
queue wait time	The time between enqueue and dequeue or execution start. Queue wait time is distinct from worker-slot time.
task cost	An umbrella term for resource demand, worker occupancy, and execution overhead associated with a task attempt. Operational risk is not part of task cost.
execution overhead	Overhead inherent to executing a task attempt, such as serialization, input fetching, setup, acknowledgement, commit, cleanup, or retry bookkeeping.
profiling overhead	Overhead caused by profiling and instrumentation, including tracing, metrics, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export.
telemetry overhead	CPU, memory, allocation, network, storage, or cardinality cost introduced by telemetry collection and export.

A.7 Uncertainty, Confidence, Residuals, and Risk

Table 10: Uncertainty, confidence, residual, and risk terms.

Term	Definition
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
entropy	A possible measure or family of measures for unpredictability. The term should only be used after specifying what distribution, signal, or behavior process is being measured.
confidence	The degree of trust in an estimate, behavior profile, graph edge, feature mapping, or model assumption. Confidence is distinct from uncertainty.
residual error	The part of observed behavior not explained by the current model estimate. Residual error exposes missing features, hidden influences, drift, or model mismatch.
prediction error	The difference between predicted and observed value for a resource, occupancy, duration, outcome, or risk-relevant dimension.
drift	A change in task behavior, feature mapping, graph relationship, context, or profile over time. Drift is a source of uncertainty and reduced confidence.
variance	The statistical spread of observed values. Variance is a variability signal but is not identical to entropy.
tail ratio	A ratio comparing a tail quantile with a central quantile, such as p95/p50 or p99/p50. Tail ratios help identify high-tail-risk behavior.
multimodality	The presence of multiple behavioral regimes or modes in the observed behavior of a task kind or task family.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is separate from task cost.
consequence	The severity of the effect caused by an incorrect estimate or control decision, such as delayed processing, queue starvation, SLO violation, replication lag, or cascading overload.
blast radius	The scope affected by an error or overload event, such as a single task, queue, worker pool, tenant, replica set, dependency, cluster, or control plane.
SLO violation risk	The operational risk that a task attempt, task kind, queue, tenant, or service will violate a service-level objective.
queue starvation risk	The risk that long-running or high-occupancy tasks delay or prevent other tasks from receiving execution capacity.

Term	Definition
retry amplification risk	The risk that retries increase work, dependency pressure, queue occupancy, or worker occupancy enough to worsen overload.
cascade overload risk	The risk that local overload propagates to other queues, workers, dependencies, tenants, or control loops.

A.8 Task Behavior Graphs

Table 11: Task Behavior Graph terms.

Term	Definition
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph is not assumed to prove causality by itself; it makes modeled influence assumptions explicit.
graph node	An element of a Task Behavior Graph representing a canonical feature, output, dimension, intermediate signal, or model concept. Each node belongs to one or more influence domains.
graph edge	A modeled influence relationship between two graph nodes. A graph edge is not a proven causal link unless validated by additional evidence.
influence role	The semantic role of a node or edge in the graph, such as baseline demand, contextual amplification, additive overhead, symptom, outcome link, risk propagation, uncertainty indicator, or profiling trigger.
baseline demand edge	An edge that connects an intrinsic feature to a baseline resource-demand or occupancy estimate. For example, payload size may contribute to network I/O or disk I/O baseline demand.
contextual amplification edge	An edge by which an execution-context factor amplifies or suppresses demand, occupancy, uncertainty, or risk.
additive overhead edge	An edge that represents additional fixed or variable overhead, such as setup, serialization, acknowledgement, commit, or retry bookkeeping.
symptom edge	An edge that connects an influence to a symptom or observed effect, such as network degradation contributing to retries or longer wall-clock duration.
outcome link	An edge that connects modeled behavior to an outcome, such as timeout, failure, success, cancellation, or dead-lettering.

Term	Definition
risk propagation edge	An edge that connects demand, occupancy, uncertainty, or outcome to an operational-risk dimension.
uncertainty indicator	A node or edge role indicating that a signal changes uncertainty or confidence rather than directly changing resource demand.
sensitivity	The estimated strength of a relationship between graph nodes. Sensitivity may be expert-defined, learned, calibrated, or represented as a qualitative level.
relevance	Whether a feature or relationship matters for a task kind, task family, or behavior profile. Relevance is distinct from sensitivity.
graph evaluation	The process of applying a Task Behavior Graph to canonical features and execution context in order to estimate demand, occupancy, uncertainty, residuals, and risk.
graph template	A reusable graph structure that can be specialized for multiple related task kinds within a task family.
model assumption	An explicit assumption encoded in the graph, feature mapping, or policy. Model assumptions should be validated, calibrated, or revised when observations contradict them.

A.9 Profiling and Observation Policy

Table 12: Profiling and observation-policy terms.

Term	Definition
profiling	The process of collecting, interpreting, and using observations to estimate task behavior. In this paper, profiling is broader than CPU profiling.
task profiling	Profiling logical task attempts in queue-worker systems in order to estimate resource demand, worker occupancy, uncertainty, and operational risk.
observation strategy	A decision about what to observe, where to observe it, how often to observe it, and under what overhead budget.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context.
accounting	Recording basic execution facts, such as start time, finish time, status, duration, input size, output size, resource usage, or occupancy.
minimal accounting	A low-overhead accounting level that records only essential task facts, such as start time, finish time, status, and basic size information.

Term	Definition
detailed accounting	An accounting level that records more detailed resource, occupancy, stage, dependency, and outcome information.
tracing	Capturing execution structure and timing through traces and spans. Tracing is an observation method, not a behavior model by itself.
sampled tracing	Tracing only a subset of tasks, attempts, or traces in order to control overhead.
tail-aware profiling	A profiling strategy that preferentially observes slow, failed, high-risk, or tail-heavy attempts.
diagnostic profiling	Deep and usually expensive profiling used to investigate unexplained behavior, high residual error, drift, or model failure.
isolated execution profiling	Profiling in a more controlled or isolated execution environment in order to reduce interference or better attribute behavior.
cooperative instrumentation	Instrumentation in which worker or task code explicitly emits structured observations such as spans, timers, byte counts, record counts, dependency calls, stage boundaries, and domain-specific signals.
sampling	Selecting a subset of tasks, attempts, events, spans, traces, or profile samples for observation or retention.
head sampling	A trace-sampling strategy that decides whether to sample before the full trace is observed.
tail sampling	A trace-sampling strategy that decides whether to sample after seeing all or most of a trace.
observation point	A point in the task lifecycle where observations can be collected, such as enqueue, dequeue, reserve, start, dependency call, retry, commit, acknowledge, or finish.
profiling budget	The allowed overhead for profiling and telemetry, including CPU, memory, allocation, storage, network, and cardinality costs.
hot path	The latency- or throughput-sensitive path of task execution where added instrumentation overhead can affect production behavior.

A.10 Manifests, Registries, and Domain Packs

Table 13: Manifest, registry, and domain-pack terms.

Term	Definition
Task Behavior Manifest	A declarative description of the behavior model for a task kind or task family. It may describe features, domains, graph nodes, graph edges, observation requirements, profiling policy defaults, known gaps, and versioning.
manifest	A machine-readable or human-readable specification used to declare part of the task behavior model.
domain pack	An extension package that provides domain-specific metrics, feature mappings, feature descriptors, graph templates, examples, or profiling defaults.
metric descriptor registry	A registry describing raw metrics and how they can be interpreted, aggregated, and mapped to canonical features.
graph declaration	The part of a manifest that declares graph nodes, graph edges, influence roles, sensitivities, confidence, and applicability conditions.
profiling policy declaration	The part of a manifest that declares default observation strategies, required observation points, sampling rules, and profiling overhead limits.
schema	A machine-validation structure for manifests, descriptors, registries, domain packs, or graph declarations.
versioning	The practice of assigning versions to schemas, manifests, domain packs, task kinds, feature registries, or graph structures in order to manage evolution and compatibility.
compatibility	The ability of manifests, schemas, domain packs, task versions, and graph structures to work together without semantic or validation conflicts.
coverage	The set of task kinds, domains, features, metrics, observation points, or risks covered by a model, manifest, or domain pack. Coverage is not the same as global completeness.
known gap	A missing, unavailable, weak, or intentionally unsupported feature, observation, domain, or graph relationship. Known gaps should remain visible rather than being hidden inside the model.
residual visibility	The ability of the model to expose unexplained behavior through residual error, reduced confidence, or missing-feature indicators.
canonical namespace	A stable naming scheme for canonical features, resource dimensions, occupancy dimensions, and risk dimensions, such as `context.network.instability` or `occupancy.worker.slot_time`.

A.11 Queue-Worker Lifecycle Terms

Table 14: Queue-worker lifecycle terms.

Term	Definition
enqueue	The lifecycle point at which a task instance is submitted to a queue.
dequeue	The lifecycle point at which a worker takes a task instance from a queue for processing.
reserve	The lifecycle point at which a worker or worker framework claims a task instance, often through a lease, visibility timeout, or reservation mechanism.
start	The lifecycle point at which a task attempt begins execution.
input fetch	The stage in which the worker fetches external input, payload data, referenced objects, or domain state required by the task attempt.
deserialize	The stage in which the worker decodes, parses, or prepares the task payload or input data for processing.
processing stage	A logical phase inside task execution, such as validation, transformation, dependency call, write, commit, or cleanup.
dependency call	An interaction with an external service, API, database, storage system, broker, or remote dependency during task execution.
retry	A repeated execution attempt or repeated operation after failure, timeout, lease expiration, cancellation, or retryable error.
backoff	A delay before retrying a task attempt or operation. Backoff can contribute to worker occupancy if execution capacity remains held.
output write	The stage in which task output, state changes, generated data, or side effects are written.
commit	The stage in which task effects are finalized, persisted, checkpointed, or made visible.
acknowledge (ack)	The lifecycle point at which a worker confirms task completion to the queue or broker.
discard	The lifecycle outcome in which a task instance or attempt is intentionally dropped or ignored.
dead-letter	The lifecycle outcome in which a task is moved to a dead-letter queue or equivalent failure storage.
finish	The lifecycle point at which a task attempt ends, whether by success, failure, timeout, cancellation, or another terminal outcome.

Term	Definition
lease expiration	The lifecycle event in which a task reservation expires before successful acknowledgement or completion.
cancellation	The lifecycle event in which a task attempt is explicitly stopped before normal completion.

B Extended Task Taxonomy

C Manifest Schema

D Algorithm Details

E Case Study Details

F Evaluation Methodology Details

G Reproducibility Checklist